

Data Security – Image Steganography Report

Student: Jon Zylfiu

Course: Data Security

1. File Analysis

The project implements image steganography using Python and Pillow. The `embed.py` file hides a secret message inside a BMP image by modifying one bit inside a selected RGB channel. The `extract.py` file reconstructs the message using the same pseudo-random pixel order generated from the shared key.

2. Capacity Calculations

The assignment stores one bit per pixel because only one colour plane is modified. Capacity therefore equals the number of pixels.

3. Message Size Calculation

$\text{message_bits} = 14 + 7 \times 1488 = 10430 \text{ bits}$

4. Visual Effects

Using `bit_position = 7` causes almost invisible changes because only the least significant bit is modified. Using `bit_position = 0` causes severe visible corruption because the most significant bit changes.

5. Wrong Key Behaviour

Extraction fails when the wrong key is used because the shuffled pixel order changes, producing meaningless output.

6. Reflection

This system is difficult to detect visually because only one low-significance bit is modified. Weaknesses include vulnerability to compression and lack of encryption.

Image	Pixels	Capacity (bits)	Capacity (bytes)
dice.bmp	181,600	181,600	22,700
flowers.bmp	150,000	150,000	18,750
tiger.bmp	171,600	171,600	21,450

7. Screenshot – `embed.py` execution

```

from PIL import Image
import random

key = 2026

# 0 = red, 1 = green, 2 = blue
colourPlane = 1

# least sig bit
significantBit = 7

coverImage = "f covert.bmp"
secretFile = "secret.txt"

outputImage = "stego_image.bmp"

image = Image.open(coverImage).convert("RGB")
dimensions = image.size
pixels = image.load()

with open(secretFile, "r", encoding="utf-8") as f:
    secret = f.read()

total_pixels = dimensions[0] * dimensions[1]

shifts = ''.join(format(ord(char), 'b').zfill(7) for char in secret)
bits = format(int(secret), 'b').zfill(14)

bits = bits + shifts

if len(bits) > total_pixels:
    raise Exception("Image does not have enough capacity")

shuffledIndices = list(range(total_pixels))
random.seed(key)
random.shuffle(shuffledIndices)

def modify_pixel(pixel, plane, bit, modifier):
    m = modifier * (2 ** (7 - bit))

    r = pixel[0] * m if plane == 0 else pixel[0]
    g = pixel[1] * m if plane == 1 else pixel[1]
    b = pixel[2] * m if plane == 2 else pixel[2]

    return (r, g, b)

for i in range(len(bits)):
    x = shuffledIndices[i % dimensions[0]]
    y = shuffledIndices[i // dimensions[0]]
    p = format(pixels[x, y][colourPlane], 'b').zfill(8)

    if p[significantBit] == '0' and bits[i] == '1':
        pixels[x, y] = modify_pixel(
            pixels[x, y],
            colourPlane,
            significantBit,
            1
        )
    elif p[significantBit] == '1' and bits[i] == '0':
        pixels[x, y] = modify_pixel(
            pixels[x, y],
            colourPlane,
            significantBit,
            -1
        )

image.save(outputImage)

print("Embedding complete. Output saved as:", outputImage)
Embedding complete. Output saved as: stego_image.bmp

```

8. Screenshot – extract.py execution

```

from PIL import Image
import random

key = 2026

# 0 = red, 1 = green, 2 = blue
colourPlane = 1

# least sig bit
significantBit = 7

stegoImage = "stego_image.bmp"

image = Image.open(stegoImage).convert("RGB")
dimensions = image.size
pixels = image.load()

total_pixels = dimensions[0] * dimensions[1]

shuffledIndices = list(range(total_pixels))

random.seed(key)
random.shuffle(shuffledIndices)

extractedBits = []

for i in shuffledIndices:
    x = i % dimensions[0]
    y = i // dimensions[0]

    p = format(pixels[x, y][colourPlane], 'b').zfill(8)

    extractedBits.append(p[significantBit])

extractedLengthBits = extractedBits[:14]
extractedLength = int(''.join(extractedLengthBits), 2)

extractedSecretASCII = []

for i in range(extractedLength):
    a = 0

    for j in range(7):
        a += int(extractedBits[14 + i * 7 + j]) * (2 ** (6 - j))

    extractedSecretASCII.append(chr(a))

secret = ''.join(extractedSecretASCII)
print(secret)

```

... Meet me at the lab after the lecture. This message is hidden using image steganography.